Intelligent Agents

Lecturer: Dr.Nguyen Thanh Binh

Outline

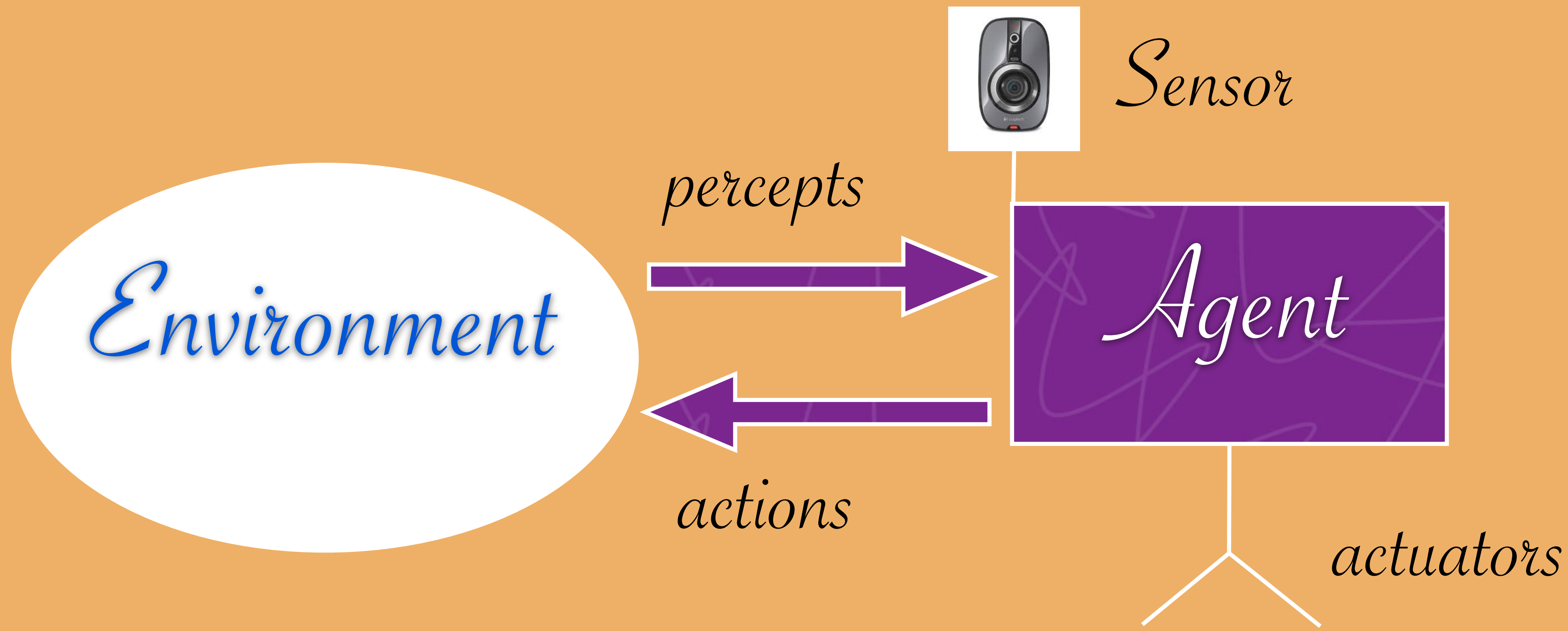
- Agents and environments
- Rationality
- PEAS model
- Environment types
- Agent types

Agents

- An agent is anything that can be viewed as perceiving its environment through sensors and acts upon that environment through actuators.
- Human agent: eyes, ears, skin and other organs for sensors; hands, legs, mouth and other body parts for actuators.

Agents

- Robot agents: cameras and infrared range finders for sensors; different motors for actuators.



percepts

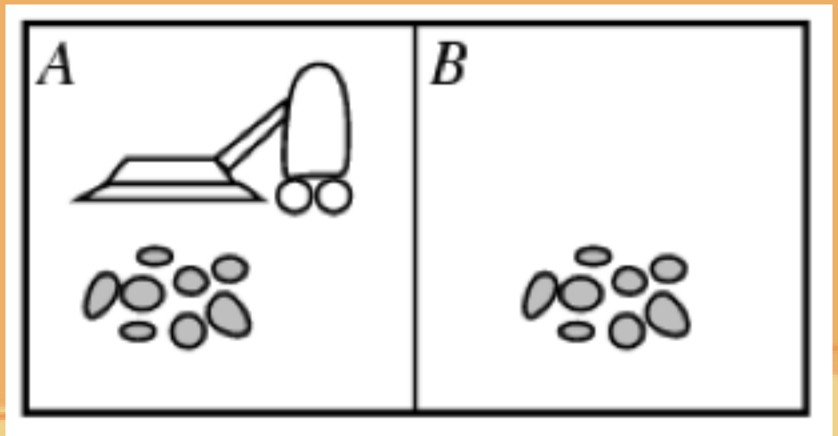
Agents

- One can consider the agent function maps from its percept history to actions.
- The agent program runs on the physical architecture to produce f.

Agent = architecture + program

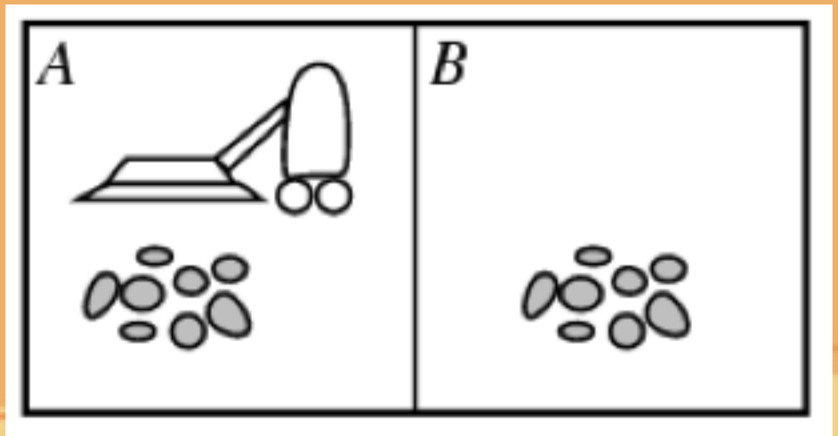
Vacuum-cleaner world

- The particular world has two locations: squarea A and B.
- The vacuum agent: perceive which square it is in and whether there is dirt in the square.



Vacuum-cleaner world

- The agent function: if the current square is dirty, move right and suck up the dirt, otherwise move to another square.
- Percepts: locations and contents, e.g [A,Dirty].
- Action: left, right, suck, and NoOp.



Vacuum-cleaner world

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
$\vdots$	$\vdots$

Rational Agents

- A rational agent should make the great efforts to "do the right thing", based on what it can perceive and the actions it can perform.
- The right action is the one that will cause the agent to be the most successful.
- Performance measure: An objective criterion for success of an agent's behaviors.

Rational Agents

- For example: performance measure of a vacuum-cleaner can be proposed by the amount of dirt cleaned up, the amount of time taken, the amount of electricity consumed,…
- One should design performance measure according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.

Rational Agents

- For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rational Agents

- Rationality is different from omniscience (all knowing with infinite knowledge).
- Agents can perform actions in order to modify future percepts so as to obtain useful information (information gathering, exploration).
- An agent is autonomous if its behavior is determined by its own experience (with ability to learn and adapt).

PEAS

- PEAS: Performance Measure, Environment, Actuators, Sensors.
- Must first specify the setting for intelligent agent design.
- Example: design an automated taxi-driver:
  - Performance measure
  - Environment
  - Actuators
  - Sensors

PEAS

- Design an automated taxi driver:
  - Performance measure: Safe, fast, legal, comfortable trip,maximize profits.
  - Environment: roads, other traffic, pedestrians, customers, police.
  - Actuators: steering wheel, accelerator, brake, signal, horn
  - Sensors: cameras, sonar, speedometer, GPS, engine sensors, keyboards.

PEAS

- Agent: Medical Diagnosis System:
  - Performance measure: healthy patients, minimize costs, lawsuits.
  - Environment: patients, hospitals and staffs.
  - Actuators: Screen display (questions, tests, diagnoses, treatments, referrals).
  - Sensors: keyboard (entry of symptoms, findings, patient's answers).

PEAS

- Agent: Part-picking robots.
  - Performance measure: percentage of parts in correct bins.
  - Environments: conveyor belt with parts and bins.
  - Actuators: jointed arms and hands.
  - Sensors: Camera, joint angle sensors.

PEAS

- Agent: Interactive English tutor:
  - Performance measure: maximized student's score on test.
  - Environment: the set of students.
  - Actuators: Screen display (exercises, suggestions, corrections).
  - Sensors: keyboard

PEAS

- Agent: Satellite image analysis system:
  - Performance measure: correct image categorization.
  - Environment: downlink from orbiting satellite.
  - Actuators: display categorization of scene.
  - Sensors: color pixel arrays.

Environment types

- Fully observable (vs. partially observable): an agent's sensors give it access to the complete state of the environment at each point in time.
- Deterministic (vs. stochastic): the next state of the environment is completely determined by the current state and the action executed by the agent. (If the environment is deterministic except for the actions of other agents, the environment is strategic.

Environment types

- Episodic (vs. sequential): in an episodic task environment, the agent's experience is divided into atomic episodes.Each episode consists of the agent perceiving and then performing a single action. The choice of action in each episode depends only the episode itself.In sequential environment, the current decision could affect all future decisions, such as chess and taxi driving.

Environment types

- Static (vs. dynamic): the environment is unchanged while the agent is deliberating. Taxi driving is dynamic: the other cars and taxi itself keep moving while the driving algorithm decides what to do next. Crossword puzzles are static. The environment is semidynamic if the environment itself does not change with the passage of time by the agent's performance score does. For example: chess when played with a clock is semidynamic.

Environment types

- Discrete (vs. continuous): A limited number of distinct, clearly defined percepts and actions. For example, chess has a discrete set of percepts and actions. Taxing driving is a continuous state and continuous-time problem.

Environment types

- Single agent (vs. multi-agent): an agent operating by itself in an environment. For example, an agent solving a crossword puzzle by itself is clearly in a single-agent environment, whereas an agent playing chess is a two-agent environment.

Environment types

- The environment type largely determines the agent design.
- The real world is partially observable, stochastic, sequential, dynamic, continuous, multi-agent.

	Chess with a clock	Chess without a clock	Taxi driving
Fully observable	Yes	Yes	No
Deterministic	Strategic	Strategic	No
Episodic	No	No	No
Static	Semi	Yes	No
Discrete	Yes	Yes	No
Single agent	No	No	No

Agent functions and programs

- An agent is completely specified by the agent function mapping percept sequences to actions.
- The job of AI is to design the agent program that implements the agent function mapping percept to actions.
- One agent function is rational.
- Aim: find a way to implement the rational agent function concisely.

Agent programs

- Take the current percept as input from the sensors (since nothing more is available from the environment) and return an action to the actuators.
- Different from the agent function that takes the entire percept history.
- If the agent's actions depend on the entire percept sequence, the agent has to remember the percepts.

Table-Driven Agent

- To keep track of the percept sequence and then use it to index into a table of actions to decide what to do next.
- To build a rational agent in this way, one must construct a table that contains the appropriate action for every possible percept sequence.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
   static: percepts, a sequence, initially empty
           table, a table of actions, indexed by percept sequences, initially fully specified

   append percept to the end of percepts
   action ← LOOKUP(percepts, table)
   return action
```

Table-driven agent

- Cons:
  - Huge table
  - Take a long time to build the table
  - No autonomy
  - Need a long time to learn the table entries

Vacuum-cleaner agent

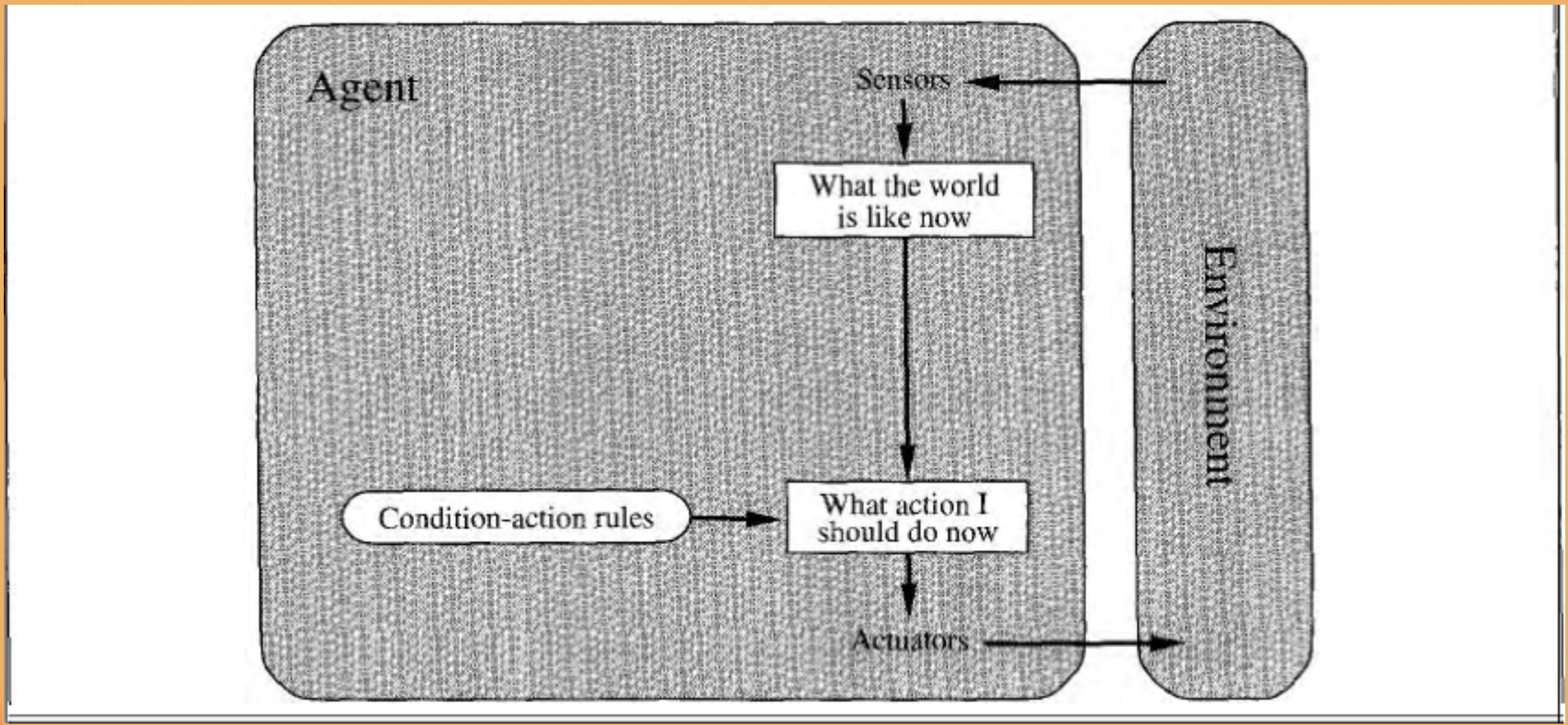
function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = **B then return Left**

Algent programs

- Four basic kinds of agent program:
  - Simple reflex agents
  - Model-based reflex agents
  - Goal-bases agents
  - Utility-bases agents

Simple reflex agent

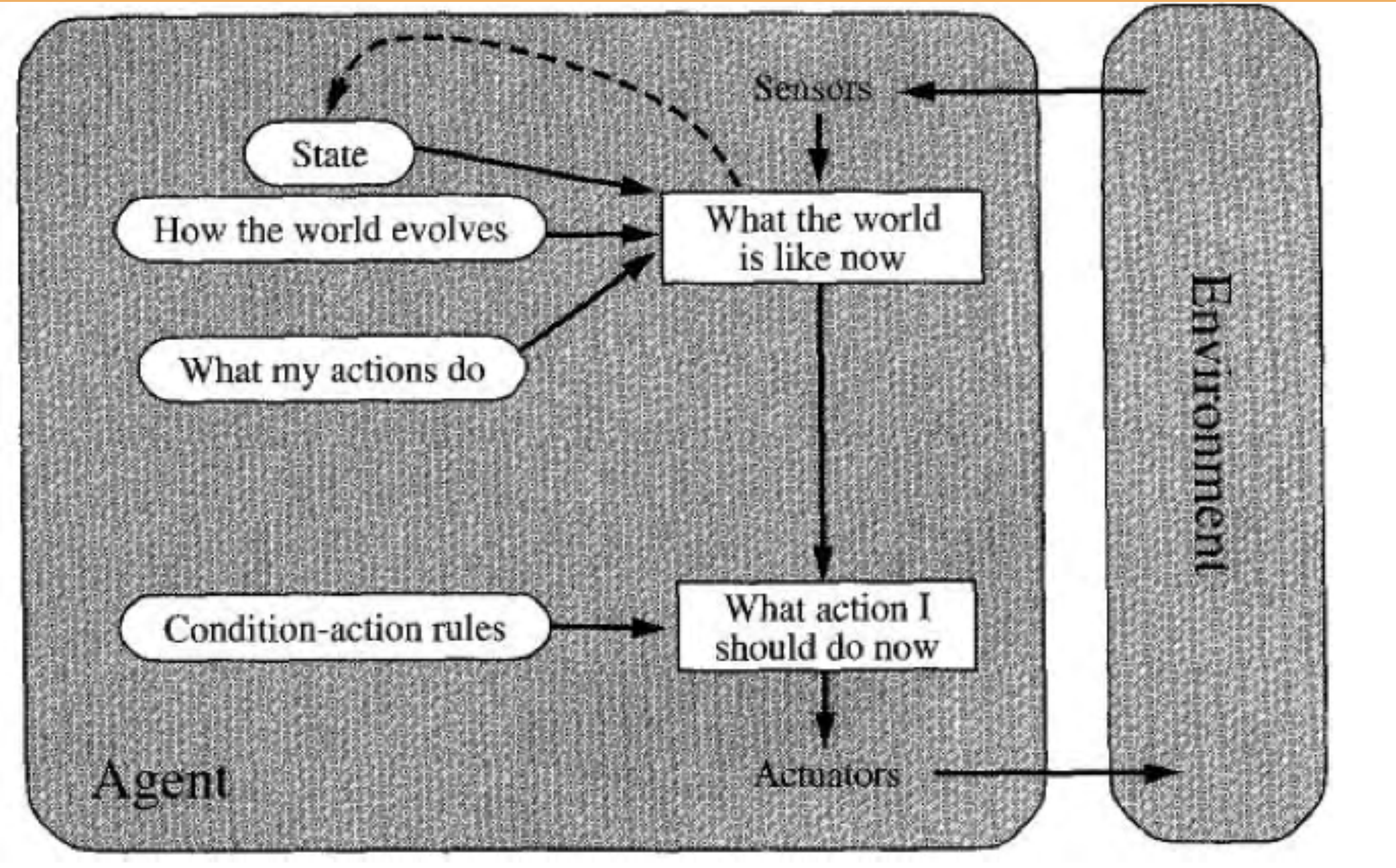


Simple reflex agent

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
   static: rules, a set of condition–action rules

   state ← INTERPRET-INPUT(percept)
   rule ← RULE-MATCH(state, rules)
   action ← RULE-ACTION[rule]
   return action
```

Model-based reflex agents

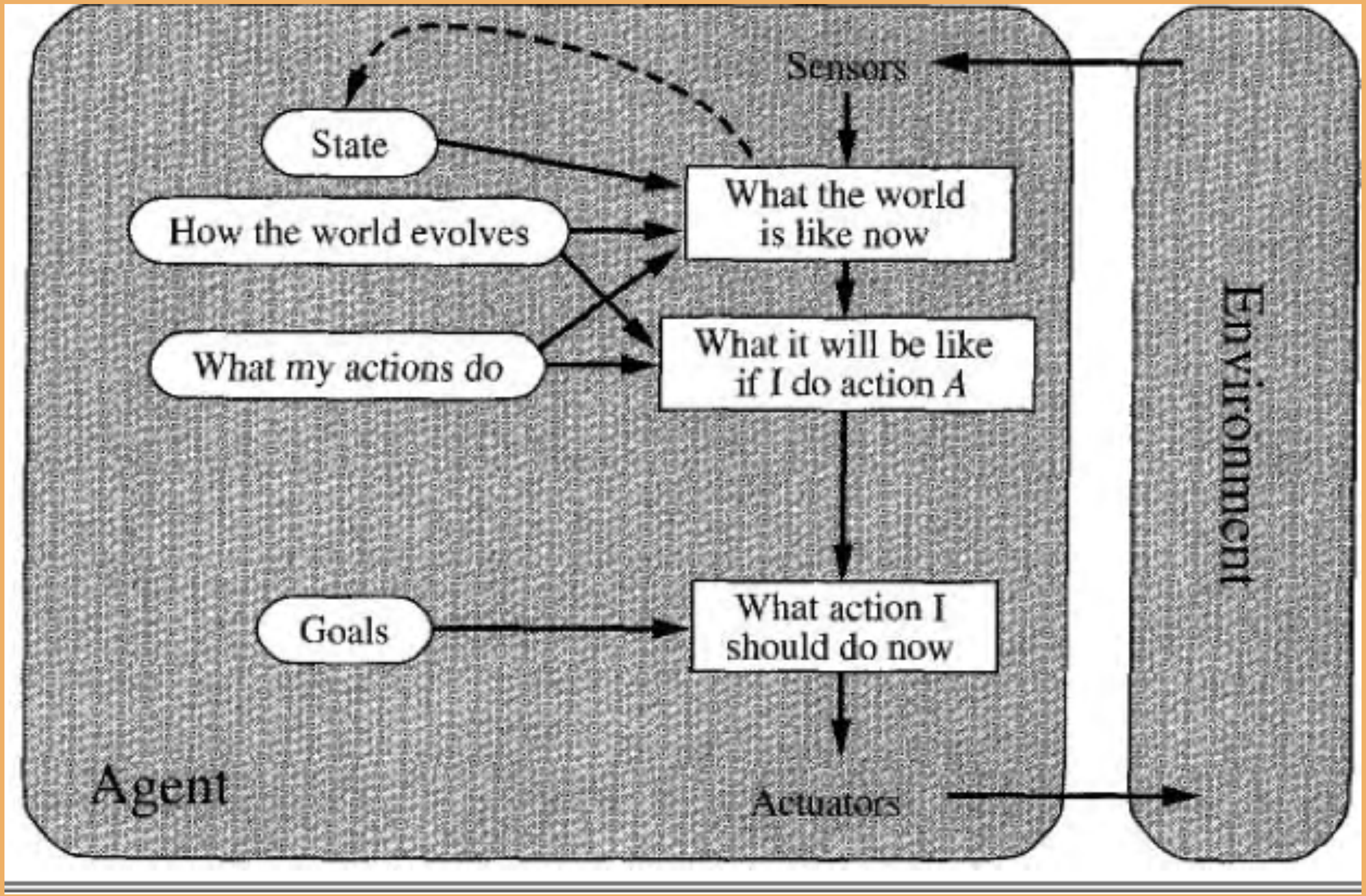


Model-based reflex agents

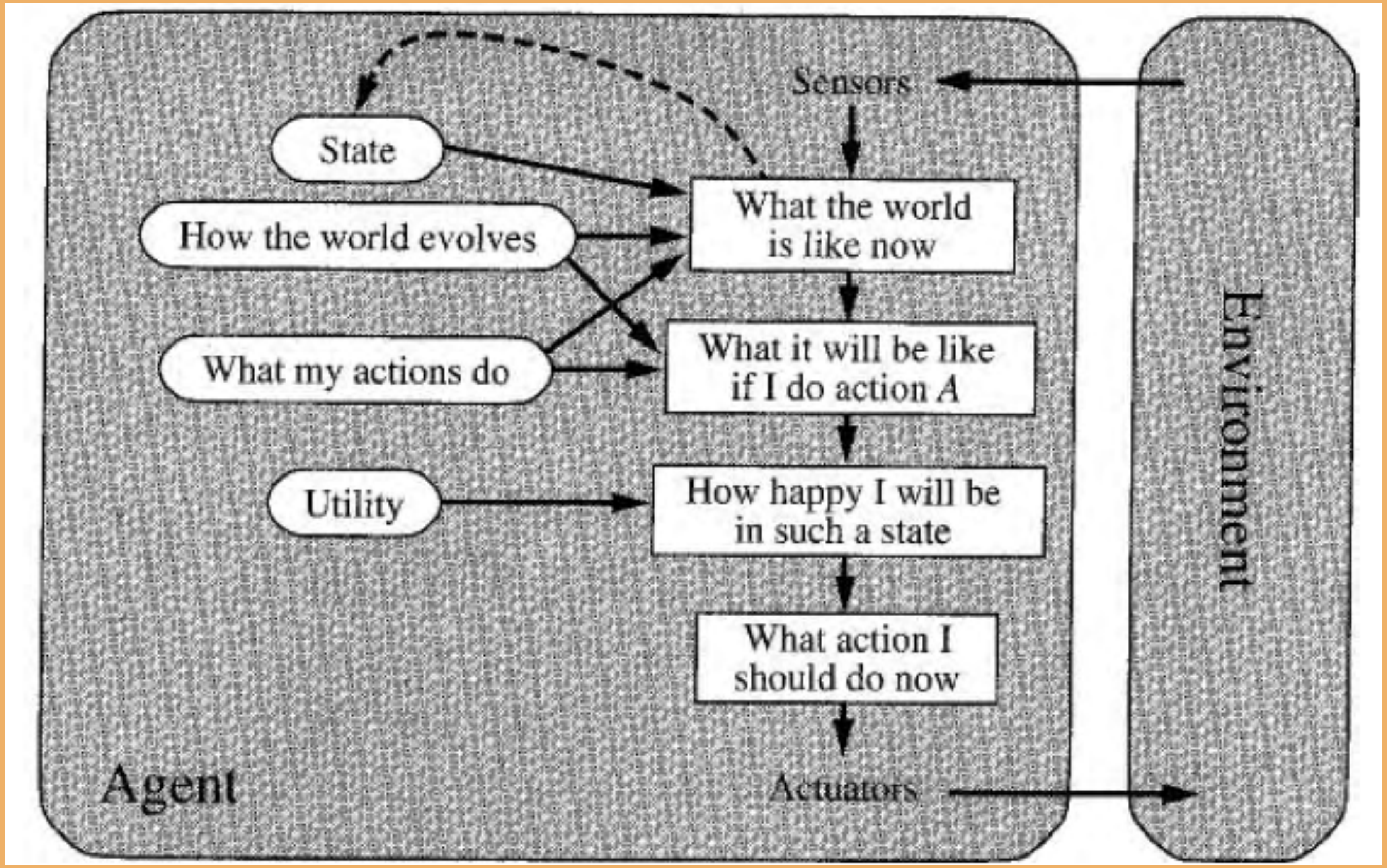
```
function REFLEX-AGENT-WITH-STATE(percept) returns an action
   static: state, a description of the current world state
           rules, a set of condition–action rules
           action, the most recent action, initially none

   state ← UPDATE-STATE(state, action, percept)
   rule ← RULE-MATCH(state, rules)
   action ← RULE-ACTION[rule]
   return action
```

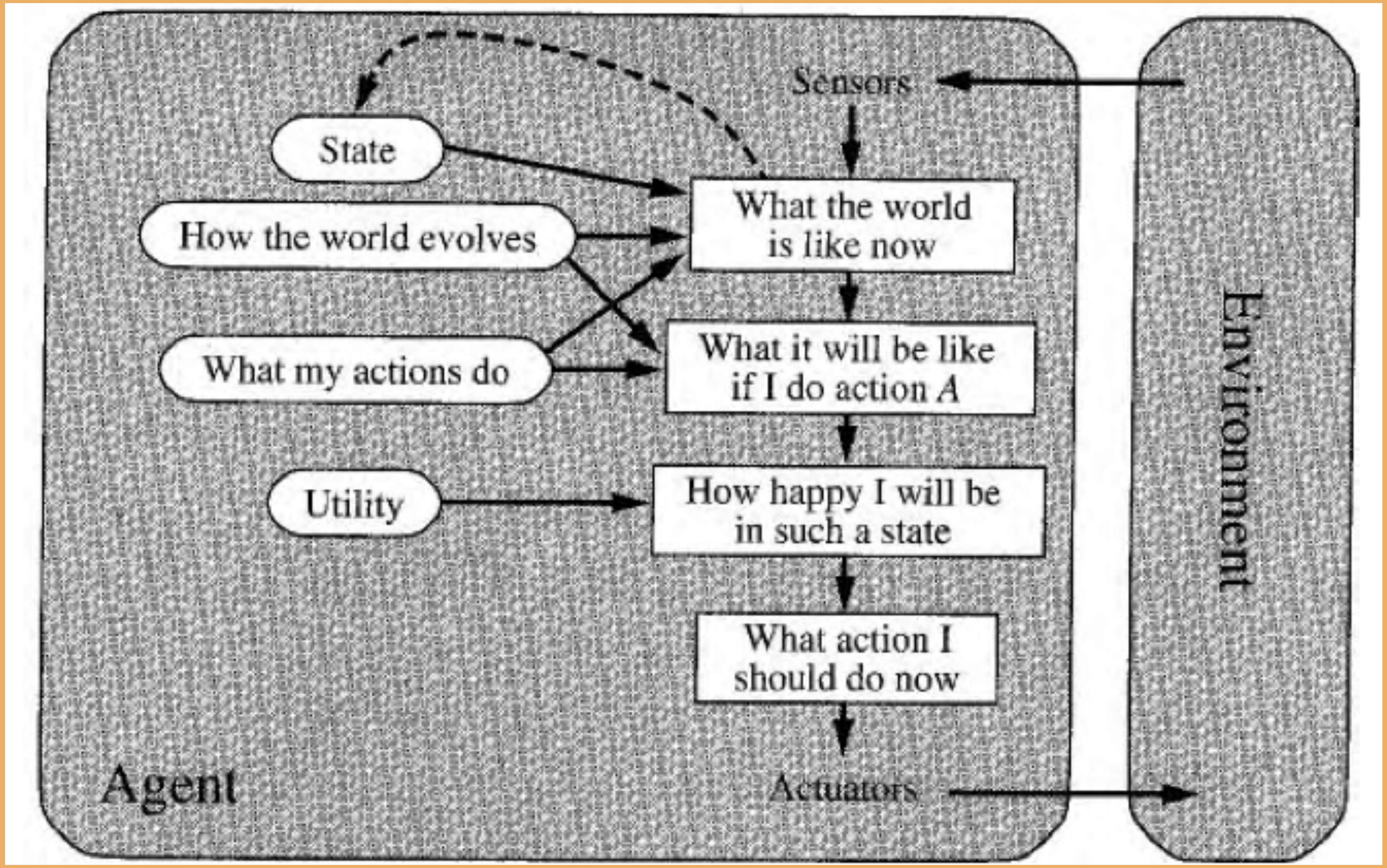
Goal-based agent



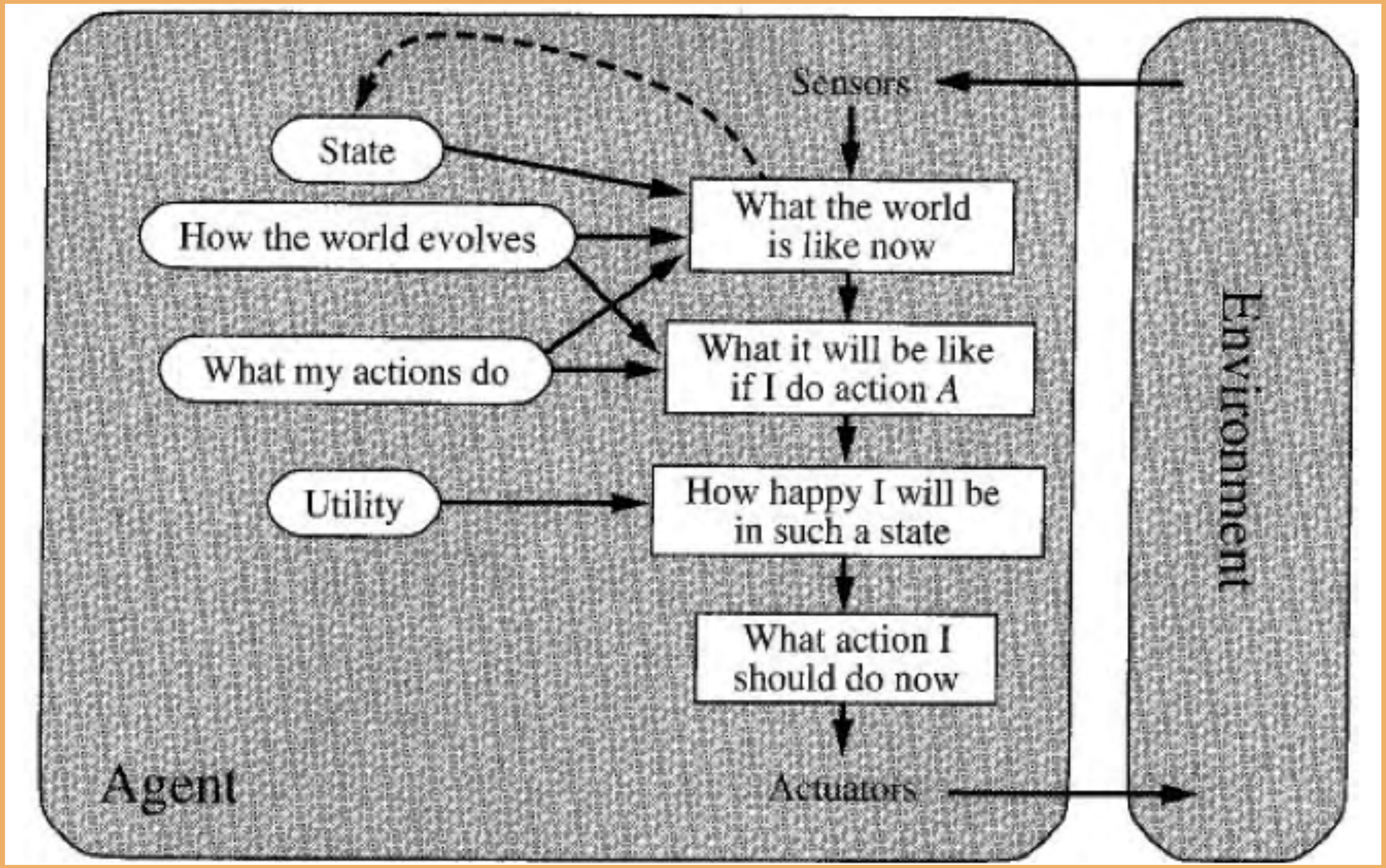
Utility-based agents



Goal-based agents



Utility-based agents



Learning agents